

Говоров Павел Игоревич К3240

ЛАБОРАТОРНАЯ РАБОТА 6.1

ВВЕДЕНИЕ В СУБД MONGODB. УСТАНОВКА MONGODB. НАЧАЛО РАБОТЫ С БД

Выполнение практического задания

```
C:\Windows\System32>mongosh
Current Mongosh Log ID: 6a16cd588522787634abc113
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB:      8.3.2
Using Mongosh:       2.8.3
```

```
test> use learn
switched to db learn
learn> show dbs
admin    40.00 KiB
config   36.00 KiB
local    76.00 KiB
learn> db.unicorns.insertOne({name: 'Aurora', gender: 'f', weight: 450})
{
  acknowledged: true,
  insertedId: ObjectId('6a16db428522787634abc114')
}
learn> show collections
unicorns
learn>

learn> show collections
unicorns
learn> db.unicorns.renameCollection("ponies")
{ ok: 1 }
learn> show collections
ponies
learn> db.ponies.drop()
true
learn> show collections

learn> db
learn
learn> db.dropDatabase()
{ ok: 1, dropped: 'learn' }
learn> show dbs
admin    40.00 KiB
config   72.00 KiB
local    76.00 KiB
```

ЛАБОРАТОРНАЯ РАБОТА 6.2

Работа с БД в СУБД MongoDB

Переключаемся на “learn” и вставляем данные

1) способ

```
learn> db.unicorns.insert({name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a16f5de8522787634abc12b') }
}
learn> db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a16f5e58522787634abc12c') }
}
learn> db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a16f5ec8522787634abc12d') }
}
learn> db.unicorns.insert({name: 'Roooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a16f5f18522787634abc12e') }
}
learn> db.unicorns.insert({name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a16f5f78522787634abc12f') }
}
learn> db.unicorns.insert({name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a16f5fd8522787634abc130') }
}
learn> db.unicorns.insert({name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a16f6028522787634abc131') }
}
learn> db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a16f6088522787634abc132') }
}
learn> db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a16f60f8522787634abc133') }
}
learn> db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a16f6158522787634abc134') }
}
learn> db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a16f6248522787634abc135') }
}
```

2) через документ

```
learn> document = ({name: 'Pavel', loves: ['apple', 'orange'], weight: 100, gender: 'm', vampires: 5})
{
  name: 'Pavel',
  loves: [ 'apple', 'orange' ],
  weight: 100,
  gender: 'm',
  vampires: 5
}
learn> db.unicorns.insert(document)
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a16f6d18522787634abc136') }
}
learn>
```

Проверка содержимого

```
learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a16f5de8522787634abc12b'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a16f5e58522787634abc12c'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a16f5ec8522787634abc12d'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a16f5f18522787634abc12e'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
]
```

2.2 ВЫБОРКА ДАННЫХ ИЗ БД

2.2.1

Списки самцов и самок, отсортированные по алфавиту и взяты первые 3

```
learn> db.unicorns.find({gender: 'm'}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a16f5de8522787634abc12b'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a16f6028522787634abc131'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a16f6d18522787634abc136'),
    name: 'Pavel',
    loves: [ 'apple', 'orange' ],
    weight: 100,
    gender: 'm',
    vampires: 5
  }
]
```

```
learn> db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a16f5e58522787634abc12c'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a16f5fd8522787634abc130'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a16f60f8522787634abc133'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
```

2) Самки, любящие carrot через findOne и limit(1)

```
learn> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
{
  _id: ObjectId('6a16f5e58522787634abc12c'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
```

```
learn> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
[
  {
    _id: ObjectId('6a16f5e58522787634abc12c'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

2.2.2

Без параметров предпочтения и пола

```
learn> db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a16f5de8522787634abc12b'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6a16f6028522787634abc131'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('6a16f6d18522787634abc136'),
    name: 'Pavel',
    weight: 100,
    vampires: 5
  }
]
```

2.2.3

Список в обратном порядке добавления

```
learn> db.unicorns.find().sort({$natural: -1})
[
  {
    _id: ObjectId('6a16f6d18522787634abc136'),
    name: 'Pavel',
    loves: [ 'apple', 'orange' ],
    weight: 100,
    gender: 'm',
    vampires: 5
  },
  {
    _id: ObjectId('6a16f6248522787634abc135'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a16f6158522787634abc134'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a16f60f8522787634abc133'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a16f6088522787634abc132'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a16f6028522787634abc131'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a16f5fd8522787634abc130'),
    name: 'Horny'
  }
]
```

2.1.4

Список единорогов с названием первого любимого предпочтения, исключив идентификатор

```
learn> db.unicorns.find({}, {_id: 0, name: 1, loves: {$slice: 1}})
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Roooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Pavel', loves: [ 'apple' ] }
]
```

2.3 ЛОГИЧЕСКИЕ ОПЕРАТОРЫ

2.3.1

Список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора

```
learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

2.3.2

Список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.

```
learn> db.unicorns.find({gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}}, {_id: 0})
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
```

2.3.3

Найти всех единорогов, не имеющих ключ vampires

```
learn> db.unicorns.find({vampires: {$exists: false}})
[
  {
    _id: ObjectId('6a16f6248522787634abc135'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

2.3.4

Список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении

```
learn> db.unicorns.find({gender: 'm'}, {_id: 0, name: 1, loves: {$slice: 1}}).sort({name: 1})
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pavel', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
```

3 ЗАПРОСЫ К БАЗЕ ДАННЫХ MONGODB. ВЫБОРКА ДАННЫХ. ВЛОЖЕННЫЕ ОБЪЕКТЫ. ИСПОЛЬЗОВАНИЕ КУРСОРОВ. АГРЕГИРОВАННЫЕ ЗАПРОСЫ. ИЗМЕНЕНИЕ ДАННЫХ

3.1 ЗАПРОС К ВЛОЖЕННЫМ ОБЪЕКТАМ

3.1.1

Добавляем города

```
learn> db.towns.insert({name: "Punxsutawney ", populatiuon: 6200, last_sensus: ISODate("2008-01-31"), famous_for: [""], mayor: {name: "Jim Wehrle"}})
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a16fc268522787634abc137') }
}
learn> db.towns.insert({name: "New York", populatiuon: 22200000, last_sensus: ISODate("2009-07-31"), famous_for: ["status of liberty", "food"], mayor: {name: "Michael Bloomberg", party: "I"}})
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a16fc2e8522787634abc138') }
}
learn> db.towns.insert({name: "Portland", populatiuon: 528000, last_sensus: ISODate("2009-07-20"), famous_for: ["beer", "food"], mayor: {name: "Sam Adams", party: "D"}})
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a16fc338522787634abc139') }
}
learn>
```

Запрос, который возвращает список городов с независимыми мэрами

```
learn> db.towns.find({"mayor.party": "I"}, {_id: 0, name: 1, mayor: 1})
[
  { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
]
```

Запрос, который возвращает список беспартийных мэров

```
learn> db.towns.find({"mayor.party": {$exists: false}}, {_id: 0, name: 1, mayor: 1})
[ { name: 'Punxsutawney ', mayor: { name: 'Jim Wehrle' } } ]
```

3.1.2

Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке и вывести результат, используя forEach

```
learn> var cursor = db.unicorns.find({gender: 'm'}).sort({name: 1}).limit(2); null;
null
learn> cursor.forEach(function(obj){ print(obj.name); })
Horny
Kenny
```

3.2 АГРЕГИРОВАННЫЕ ЗАПРОСЫ

3.2.1

Количество самок единорогов весом от полутонны до 600 кг

```
learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 600}}).count()  
2
```

3.2.2

Список предпочтений

```
learn> db.unicorns.distinct("loves")  
[  
  'apple',      'carrot',  
  'chocolate', 'energon',  
  'grape',      'lemon',  
  'orange',     'papaya',  
  'redbull',    'strawberry',  
  'sugar',      'watermelon'  
]
```

3.2.3

Количество особей единорогов обоих полов

```
learn> db.unicorns.aggregate({"$group": {_id: "$gender", count: {$sum: 1}}})  
[ { _id: 'f', count: 5 }, { _id: 'm', count: 7 } ]
```

3.3 РЕДАКТИРОВАНИЕ ДАННЫХ

3.3.1

В новой версии MongoDB нет команды save

```
learn> db.unicorns.save({name: 'Barney', loves: ['grape'], weight: 340, gender: 'm'})  
TypeError: db.unicorns.save is not a function  
learn> db.unicorns.insert({name: 'Barney', loves: ['grape'], weight: 340, gender: 'm'})  
{  
  acknowledged: true,  
  insertedIds: { '0': ObjectId('6a17015b8522787634abc13a') }  
}  
learn>
```

Проверка содержимого

```
learn> db.unicorns.find({name: 'Barney'})  
[  
  {  
    _id: ObjectId('6a17015b8522787634abc13a'),  
    name: 'Barney',  
    loves: [ 'grape' ],  
    weight: 340,  
    gender: 'm'  
  }  
]
```

3.3.2

Для самки единорога Аупа внести изменения в БД: теперь ее вес 800, она убила 51 вампира

Первый способ не сработал, использовал другой

```
learn> db.unicorns.update({name: 'Ayna'}, {name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 800, gender: 'f', vampires: 51})
DeprecationWarning: Collection.update() is deprecated. Use updateOne, updateMany, or bulkWrite.
MongoInvalidArgumentError: Update document requires atomic operators
learn> db.unicorns.update({name: 'Ayna'}, {$set: {weight: 800, vampires: 51}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> _
```

```
learn> db.unicorns.find({name: 'Ayna'})
[
  {
    _id: ObjectId('6a16f5fd8522787634abc130'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  }
]
```

3.3.3

Для самца единорога Raleigh внести изменения в БД: теперь он любит рэдбул и проверить

```
learn> db.unicorns.update({name: 'Raleigh'}, {$set: {loves: ['redbull']}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Raleigh'})
[
  {
    _id: ObjectId('6a16f6088522787634abc132'),
    name: 'Raleigh',
    loves: [ 'redbull' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
learn> _
```

3.3.4

Всем самцам единорогов увеличить количество убитых вампиров на 5 и проверить содержимое коллекции unicorns

```
learn> db.unicorns.update({gender: 'm'}, {$inc: {vampires: 5}}, {multi: true})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 8,
  modifiedCount: 8,
  upsertedCount: 0
}
learn> db.unicorns.find({gender: 'm'})
[
  {
    _id: ObjectId('6a16f5de8522787634abc12b'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 68
  },
  {
    _id: ObjectId('6a16f5ec8522787634abc12d'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 187
  },
  {
    _id: ObjectId('6a16f5f18522787634abc12e'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 104
  }
]
```

3.3.5

Изменить информацию о городе Портланд: мэр этого города теперь беспартийный и проверить содержимое коллекции towns

```
learn> db.towns.update({name: "Portland"}, {$unset: {"mayor.party": 1}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.towns.find({name: "Portland"})
[
  {
    _id: ObjectId('6a16fc338522787634abc139'),
    name: 'Portland',
    populatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
```

3.3.6

Изменить информацию о самце единорога Pilot: теперь он любит и шоколад и проверить содержимое коллекции unicorns

```
learn> db.unicorns.update({name: 'Pilot'}, {$push: {loves: 'chocolate'}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Pilot'})
[
  {
    _id: ObjectId('6a16f6158522787634abc134'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
```

3.3.7

Изменить информацию о самке единорога Aurora: теперь она любит еще и сахар, и лимоны и проверить содержимое коллекции unicorns

```
learn> db.unicorns.update({name: 'Aurora'}, {$addToSet: {loves: {$each: ['sugar', 'lemon']}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Aurora'})
[
  {
    _id: ObjectId('6a16f5e58522787634abc12c'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

3.4 УДАЛЕНИЕ ДАННЫХ ИЗ КОЛЛЕКЦИИ

3.4.1

Удалите документы с беспартийными мэрами и проверьте содержание коллекции

```
learn> db.towns.remove({"mayor.party": {$exists: false}})
DeprecationWarning: Collection.remove() is deprecated. Use deleteOne, deleteMany, findOneAndDelete, or bulkWrite.
{ acknowledged: true, deletedCount: 2 }
learn> db.towns.find()
[
  {
    _id: ObjectId('6a16fc2e8522787634abc138'),
    name: 'New York',
    populatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  }
]
learn>
```

Очистите коллекцию и просмотрите список доступных коллекций

```
learn> db.towns.remove({})
{ acknowledged: true, deletedCount: 1 }
learn>

learn> show collections
towns
unicorns
learn>
```

4.1 ССЫЛКИ В БД

Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание

```
learn> db.zones.insert({_id: "forest", name: "Uniforest", description: "Лес для единорогов"})
{ acknowledged: true, insertedIds: { '0': 'forest' } }
learn> db.zones.insert({_id: "river", name: "Uniriver", description: "Река для единорогов"})
{ acknowledged: true, insertedIds: { '0': 'river' } }
learn> db.zones.insert({_id: "mountain", name: "Unimountain", description: "Гора для единорогов"})
{ acknowledged: true, insertedIds: { '0': 'mountain' } }
learn> db.zones.find()
[
  { _id: 'forest', name: 'Uniforest', description: 'Лес для единорогов' },
  { _id: 'river', name: 'Uniriver', description: 'Река для единорогов' },
  {
    _id: 'mountain',
    name: 'Unimountain',
    description: 'Гора для единорогов'
  }
]
learn>
```

Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания

```
learn> db.unicorns.update({name: 'Leia'}, {$set: {zone: {$ref: "zones", $id: "river"}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.update({name: 'Aurora'}, {$set: {zone: {$ref: "zones", $id: "forest"}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.update({name: 'Unicrom'}, {$set: {zone: {$ref: "zones", $id: "mountain"}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> _
```

Проверка

```
learn> db.unicorns.find({zone: {$exists: true}})
[
  {
    _id: ObjectId('6a16f5e58522787634abc12c'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43,
    zone: DBRef('zones', 'forest')
  },
  {
    _id: ObjectId('6a16f5ec8522787634abc12d'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 187,
    zone: DBRef('zones', 'mountain')
  },
  {
    _id: ObjectId('6a16f60f8522787634abc133'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33,
    zone: DBRef('zones', 'river')
  }
]
```

4.2 НАСТРОЙКА ИНДЕКСОВ

4.2.1

Проверьте, можно ли задать для коллекции unicorns индекс для ключа name с флагом unique

```
learn> db.unicorns.ensureIndex({name: 1}, {unique: true})
[ 'name_1' ]
learn>
```

4.3 УПРАВЛЕНИЕ ИНДЕКСАМИ

4.3.1

Получите информацию о всех индексах коллекции unicorns

```
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn>
```

Удалите все индексы, кроме индекса для идентификатора

```
learn> db.unicorns.dropIndexes()
{
  nIndexesWas: 2,
  msg: 'non-_id indexes dropped for collection',
  ok: 1
}
```

Попытайтесь удалить индекс для идентификатора

```
learn> db.unicorns.dropIndex("_id")
MongoServerError[InvalidOptions]: cannot drop _id index
learn>
```

4.4 ПЛАН ЗАПРОСА

4.4.1

Создать объёмную коллекцию numbers

```
learn> for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}
```

Выбрать последние четыре документа

```
learn> db.numbers.find().sort({$natural: -1}).limit(4)
[
  { _id: ObjectId('6a1709d58522787634ad47da'), value: 99999 },
  { _id: ObjectId('6a1709d58522787634ad47d9'), value: 99998 },
  { _id: ObjectId('6a1709d58522787634ad47d8'), value: 99997 },
  { _id: ObjectId('6a1709d58522787634ad47d7'), value: 99996 }
]
```

Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса

```
learn> (db.numbers.find().sort({$natural: -1}).limit(4).explain("executionStats")).executionStats.executionTimeMillis
0
```

Создайте индекс для ключа value, получите информацию о всех индексах коллекции numbers, выполните запрос 2, проанализируйте план выполнения запроса с установленным индексом

```
learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
```

```
learn> (db.numbers.find().sort({value: -1}).limit(4).explain("executionStats")).executionStats.executionTimeMillis
0
```

На моем устройстве выполнилось одинаково быстро